CD32X Builder's Addendum: Lessons from the Tyrian 32X Port

Oct 6, 2026 · @erexx

Purpose and scope

This addendum records what the Tyrian 2.1 port taught about building for the Sega CD + 32X. It complements the two Kobo documents: *Kobo Deluxe Sega CD32X – How It Boots and Runs* (cited here as **[KOBO-BOOT]**) and *Sega 32X/CD Boot Sequence and Sub-CPU Command Ledger* (**[KOBO-LEDGER]**). Those describe a disc that boots itself; Tyrian took a different route, and the differences are the main lesson.

Section numbers such as "[KOBO-BOOT] §6.3" point at the Kobo documents. Everything else comes from the Tyrian work: the cartridge build, the PC host build, and the T3 (files from CD) and T5 (saves) designs.

Status caveat. Most Tyrian results were measured in Fusion 3.64 and ares; [KOBO-BOOT] §6 warns that Fusion hides real hardware faults. The T3 and T5 designs below describe the code as handed over; check them against the final shipped Tyrian sources before relying on details. Items still unconfirmed on a real console are listed in the last section.

Two architectures: CD boot vs cartridge + CD

The choice of who coordinates decides almost everything else. Kobo boots from the disc and makes the Sub-CPU the coordinator ([KOBO-BOOT] §1). Tyrian boots from a 32X cartridge with the Sega CD attached (Mode 1) and makes the Genesis 68000 the coordinator.

Aspect	Kobo (CD boot, [KOBO-BOOT])	Tyrian (cartridge + CD, Mode 1)
Boot	BIOS → boot block → `APP.BIN` on the Sub-CPU (§3, stages 0–2)	Standard 32X cartridge boot; the 68000 starts at ROM `0x800`
Coordinator	Sub-CPU; the 68000 is a listener in RAM at `0xFF1000` (§3, stage 3)	68000 (`cart_md.s`); the Sub-CPU runs D32XR's `cd.bin` and only serves CD requests
SH2 program	Copied by the 68000 into the frame buffer; boot ROM moves it to SDRAM on `"_CD_"` (§3, stages 4–5)	Boot ROM copies only `.sdcode` + `.data` to SDRAM; `.text` runs from ROM through the cache
SDRAM for code	Program must end below `0x0600C000` (§5, §6.9)	~6 KB of code in SDRAM; ~150 KB left for `malloc`
Request path	SH2 → COMM → Sub-CPU → Sega CD comm → 68000 → hardware (§1)	SH2 → COMM → 68000 → hardware, or → Sub-CPU via D32XR's `scd_*` calls for the CD

Aspect	Kobo (CD boot, [KOBO-BOOT])	Tyrian (cartridge + CD, Mode 1)
Bulk data	Copied into the RAM cart (CMD 46) or frame buffer (CMD 50/51) at start-up	Lives in ROM and is used in place; levels streamed from CD on demand
Hardware needed	Sega CD + 32X + 4 MB RAM cart	32X cartridge; Sega CD optional (music, levels)
Slave SH2	Parked, unused (§1)	Sound effect mixer feeding the PWM

How to choose.

- Pick **cartridge + CD** when the game's working set does not fit 256 KB of SDRAM but its read-only data does fit 4 MB of ROM. Data used in place from ROM costs no RAM at all; this alone made Tyrian possible.
- Pick **CD boot** when nothing can be flashed to a cartridge, or when the data is far larger than 4 MB. Expect a RAM cart or careful frame-buffer staging, as Kobo needed.
- In both, the request path is the bottleneck: every extra CPU in the chain adds a handshake. Tyrian's shorter path (one hop for pad, sound and saves) is a real gain.

COMM registers and command numbers

The two projects share command numbers but not register meanings; merging code between them without a shared table will break both. Tyrian kept Kobo's numbers where the meaning matched (45, 48, 52–59) and added 60–63, but it reuses COMM4–COMM14 for data packets, which Kobo reserves for start-up, heartbeats and status.

Register	Kobo use ([KOBO-LEDGER], [KOBO-BOOT] §4.1)	Tyrian use	Conflict
COMM0	Command (low byte); SFX id in high byte for CMD 48	Command; cleared by the 68000 when done	None
COMM2	Command parameter	Parameter; also the `GO` flag (`0x474F`) during SRAM access	Minor
COMM4	Start-up only (`S_OK`, legacy `0x1234`)	Name, size or data word during CMD 60–63	**Yes**
COMM6	Genesis vblank heartbeat (`V`)	Name, size or data word	**Yes**
COMM8 / 10	Replies; `0x0AAA` /	Pad + tick reply;	Compatible outside

Register	Kobo use ([KOBO-LEDGER], [KOBO-BOOT] §4.1)	Tyrian use	Conflict
	`0x0BBB` handshake in COMM10	handshake; data words	streams
COMM12	SH2 status code (`STATE`); 16-bit replies	Packet sequence number (sender)	**Yes**
COMM14	Sub-CPU heartbeat (`P`)	Packet acknowledgement (receiver)	**Yes**

Command	Kobo	Tyrian
38	Open file (length in COMM8)	–
45 / 54	Play CD track / stop music	Same
46, 50, 51	Chunk → cart / frame buffer	–
48	Pad (+ SFX)	Pad (SFX run on the slave SH2 instead)
52	Genesis display on/off	Same
53, 56–59	Sound pack, backup RAM	Acknowledged with "not available"
60	–	CD find: name in COMM4–14 → size, first sector
61	–	CD read: sector, byte count → streamed
62 / 63	–	SRAM read / write, streamed

Recommendations.

- Keep one command registry for all CD32X projects and never reuse a number with a different meaning.
- Write down, per register, **who writes it in which phase**. [KOBO-BOOT] §6.2 says "one writer per register"; Tyrian shows the rule can be per phase rather than global, as long as the hand-over is defined (see the next section).
- If a project needs Kobo-style heartbeats in COMM6/COMM14, suspend them for the length of a stream, or move streaming to a different transport.

Moving bulk data over COMM

One command per word is too slow; a sequence-numbered packet stream moves 8 bytes per handshake without frame-buffer ownership switches. Kobo moves bulk data through the RAM cart or the frame buffer ([KOBO-BOOT] §3 stage 7, CMD 46/50/51); the stream is the lighter alternative when the amount per request is small (a 4 KB file block, a 2.5 KB save).

Packet format. Data in COMM4, 6, 8, 10 (four words, high byte first), then a sequence number in COMM12. The receiver reads the four words and writes the same number into COMM14. Sequence numbers run 1 to `0xFFFE`; `0xFFFF` means error from the 68000 side, or "ready to receive" when the 68000 is the receiver.

Rules that make it safe.

1. **Direction fixes ownership.** The sender owns COMM4–12, the receiver owns COMM14. Nobody else writes them during the stream.
2. **Reset before every command.** The SH2 zeroes COMM12 and COMM14 while the 68000 is idle. Otherwise a stream that ended on sequence 1 makes the next stream's first packet look already sent.
3. **Wait for idle first.** Before writing a new command, wait (with a timeout) until COMM0 is 0. A side that gave up on a stream leaves the other one timing out, and a new command written into that window is lost.
4. **Data before sequence.** The sequence number is written last, so seeing it means the data words have landed.
5. **Timeouts on both sides.** The first packet of a CD read gets seconds (spin-up, seek, 64 KB read); later packets get hundreds of milliseconds.
6. **Pause between polls**, as [KOBO-BOOT] §6.2 requires. Keep the pauses short (a few loop iterations) so the stream still reaches a few hundred KB/s.
7. **Read the command flag twice** where a torn read is possible ([KOBO-BOOT] §6.1, HW10). Tyrian's 68000 loop reads COMM0 twice before acting.

The parameters of a command share registers with the stream, so the receiver must copy all parameters out before the first data packet overwrites them.

CD data access

Read the disc in large windows and treat CD audio as interruptible, not forbidden. Kobo avoids the conflict by loading everything before the music starts ([KOBO-BOOT] §3 stage 7, status `4B08`; rule §6.3). Tyrian cannot: its levels are too big to preload, so it reads them at level transitions.

Read windows, not sectors. Each separate read command can cost a seek and part of a rotation. Tyrian's 68000 keeps a 32-sector (64 KB) window in Word RAM and serves the SH2's 4 KB block requests from it, so a level load costs one or two CD accesses. Word RAM is used in 1M mode, as in Kobo ([KOBO-BOOT] §5): the Sub-CPU reads into its bank at `0x0C0000`, which the 68000 sees at `0x600000` in Mode 1.

Find files yourself. Tyrian reads the ISO 9660 directory directly (primary volume descriptor at sector 16, then the root directory) instead of relying on a file-open command. That needs only one primitive, "read

N sectors". The same C file runs on the 68000 and in the PC test build, so the parsing is tested on every PC run.

Disc layout rules (`make_tyrian_cd.py`).

- Level-1 ISO names (8.3, upper case) in one root directory sector.
- Volume id `TYRIAN32X`, checked by the 68000: another disc in the drive is treated as "no files", not read as garbage.
- 32 spare sectors at the end of the data track, so a full window read near the end never runs off the track.
- Mixed-mode BIN/CUE built with Kobo's `make_mixed_cd.py` layout: data track 1, audio from track 2 ([KOBO-BOOT] §2).

Audio vs data. [KOBO-BOOT] §6.3 is right that the drive cannot do both. The fix is bookkeeping: after any CD data read, the SH2 marks the music as stopped. The next `play_song` then restarts its track even when it is the same song; otherwise the game believes music is playing while the drive is silent. Load at moments when the music changes anyway (level start, menu change).

RV mode and keeping the SH2s off the ROM

When the 68000 sets RV, neither SH2 may touch the cartridge, so code that runs from ROM must be moved out of the way first. Kobo uses Chilly's RV method for its cart writes (CMD 46, `CHILLY_RV_MEMCPY`, [KOBO-LEDGER]); its SH2 runs entirely from SDRAM, so the problem never arose. Tyrian's SH2 code lives in ROM, which turns RV into a coordination problem.

Tyrian uses RV for battery SRAM (saves): with RV set, the 68000 sees the cartridge at `0x000000`, and the SRAM at `0x200001` (odd bytes) once `0xA130F1` enables it. The ROM header declares 8 KB there (`"RA"`, `0xF8`, `0x20`, `0x200001`–`0x203FFF`).

The sequence.

1. The 68000 copies data between SRAM and its own RAM only. All COMM streaming happens before (write) or after (read) the RV window, never during it.
2. The routines that set RV run from Genesis RAM (`.data`, copied at start-up) with interrupts off: with RV set, the `0x880000` window and the vector jump list are gone.
3. The master SH2 parks the slave first: the slave polls a flag in cache-through SDRAM and spins in a function placed in SDRAM, interrupts masked.
4. The master enters its own SDRAM wait function, masks interrupts (its handlers are in ROM), and only then writes `GO` (`0x474F`) into COMM2.
5. The 68000 waits for `GO`, sets RV, copies, clears RV, then continues the stream.

The `GO` handshake closes a race: without it the 68000 could set RV while the master is still executing the few ROM instructions between sending the command and entering its SDRAM wait.

Rules for SDRAM wait code. No calls into ROM functions, no library calls (gcc may emit `memcpy` or `__udivsi3`), counted loops instead of the millisecond clock, and the literal pool in the same section as the function.

Saves themselves. Each file is stored twice with a generation number and a checksum. A new save overwrites the older copy and is read back; an emulator or cartridge without SRAM fails the read-back instead of appearing to save. A power loss mid-write leaves the other copy intact. This plays the role Kobo's backup-RAM commands play (CMD 56–59, [KOBO-BOOT] §4.1) when there is no Sega CD.

Running SH2 code from ROM

Running the SH2's code from ROM through its 4 KB cache works, and it is not the bottleneck, as long as nothing else hogs the cartridge bus. This frees most of SDRAM for data, which Kobo's 48 KB code limit (`0x0600C000`, [KOBO-BOOT] §5, §6.9) does not allow.

The 68000 must not busy-loop in ROM. Kobo's listener already runs from Genesis RAM at `0xFF1000` ([KOBO-BOOT] §3 stage 3). Tyrian first left its command loop in ROM: the loop's instruction fetches kept the cartridge bus busy, and every SH2 cache miss, tile read and sprite read waited. Moving the loop and its pause into RAM was a large, immediate speed-up. Calls from that loop into ROM code are fine; the constant polling is what hurts.

Measured, not assumed. A build switch put the hottest functions (enemy logic, blitters) into SDRAM. Same scenes, same moments: frame times barely changed (about 55 vs 53 ms), while the moved code took ~7 KB from the tile cache. Code fetch was not the limit; frame-buffer writes were (next sections).

Practical points.

- The boot ROM copies only `.sdcode` + `.data`; `make_tyrian_cart.py` reads entry points, VBRs and the copy size from the ELF symbols, so the header can never drift from the build.
- Interrupt handlers and anything that must run during RV belong in `.sdcode`.
- Data read from ROM must be aligned for the access size: the SH2 faults on unaligned 16/32-bit reads. Tyrian's file-system builder pads sprite sheets and tiles to 4-byte offsets.

Memory: living in 256 KB

OpenTyrian as written needs about 730 KB of RAM; the port runs in 256 KB of SDRAM by refusing to copy anything that can be read in place. Kobo solved the same pressure with a hand-placed memory map ([KOBO-BOOT] §5); Tyrian's techniques suit a large C codebase that cannot be rewritten.

Technique	What it did for Tyrian
Zero-copy from ROM (`fileMap`)	Sprites, tiles, palettes and pictures used where they sit in ROM
Tables baked at build time	Weapon and enemy tables pre-converted to the C struct layout, big-endian: ~130 KB saved
Allocate on demand, free at the title	Gameplay arrays exist only during a game; one macro line per array, no use sites changed
Frame buffer spare area as scratch	Each 128 KB buffer has ~64 KB free beside the picture; used for a second screen and a tile cache

Technique	What it did for Tyrian
Arrays carved from one block	Level arrays cut out of a 64,000-byte buffer: no `malloc`, no fragmentation
Compressed files in ROM	Independent 4 KB LZSS chunks, so a seek decodes one chunk; level data 538 → 148 KB
Sentinel for unallocated arrays	Pointers hold the odd address `1`, so a stray 16/32-bit access faults at once instead of reading boot ROM

Traps found the hard way.

- **Large locals.** A 64,000-byte local array overran the 16 KB SH2 stack into the heap and showed up as a garbage line in a menu. Guard words at the stack's end and a red "STACK OVERFLOW" screen now catch it.
- **The frame buffer swaps.** The SH2 reaches only the back buffer, and every flip swaps which physical buffer that is. Scratch data survives one present at most. The PC build emulates the swap exactly, so code that relies on stale contents shows wrong pictures there.
- **`.bss` and start-up.** Kobo warns that a long `.bss` clear once raced the start handshake ([KOBO-BOOT] §6.9). Keep `.bss` small; allocate big arrays at runtime.
- **Measure the real heap.** The cartridge builder prints the actual heap size from the ELF; the PC build reports the peak heap per phase. Budgets based on estimates were wrong more than once.

Frame buffer performance

The 32X frame buffer is the real speed limit: about 10 SH2 cycles per 16-bit access, because the video chip reads the same memory. A full 64 KB screen takes ~14 ms to write, so a 30 fps game has room for roughly two full-screen passes per frame. Tyrian's level 1 went from ~120 ms per frame (8 fps) to about 37 ms of game work (20–30 fps) by following the rules below.

Rules.

1. **Write 16 or 32 bits, never bytes.** A byte write costs a whole access. [KOBO-BOOT] §6.4 found byte writes of `0x00` did not even land in Fusion. Odd destinations are written as byte + 16-bit pairs + byte.
2. **Draw straight into the visible-to-be buffer.** Composing in RAM and copying 64 KB each frame cost ~20 ms (copy + present). Drawing the playfield directly into the back buffer removed both.
3. **Write every pixel once.** Opaque background tiles are written including colour 0, and only uncovered strips are cleared, instead of clearing the screen first.
4. **Use the SH2's DMA for opaque lines.** Compose a line in RAM, let DMA channel 0 copy it (cycle-steal, 32-bit units) while the CPU composes the next. Measured 18–19 ms vs 24 ms with the CPU. Give every DMA wait a timeout and fall back to the CPU for good if one fails.
5. **Check the compiler's output.** `memcpy(&pair, p, 2)` at a possibly odd address became a library call on the SH2 and made the background 2.4× slower. The build now disassembles the blitters and warns on any `memcpy`/`memset`/`memmove` call.

6. **A second SH2 does not help drawing.** Both CPUs share the path to the frame buffer; they would take turns. Use the slave for sound or game logic.

Frame pacing. A game that delivers frames after 2, 3 or 4 refreshes scrolls jerkily even at a decent average. Tyrian shows each frame exactly N refreshes after the last (N = the 3rd largest of the last 16 natural frame lengths) and requests the flip only after the V-blank has ended; a flip requested as the blank starts can land in that blank or the next.

Colour 0. [KOBO-BOOT] §6.5: `0x0000` is see-through to the Genesis layer. Tyrian keeps the Genesis display off during play, so its black borders in colour 0 are safe; any game that shows Genesis text must draw black as `0x0400`.

Sound and music

Put sound effects on the otherwise idle slave SH2 and music on CD audio tracks; both cost the game almost nothing. Kobo plays effects on the Sega CD's PCM chip through the Sub-CPU (CMD 53 and the SFX byte of CMD 48, [KOBO-LEDGER]) and leaves the slave parked ([KOBO-BOOT] §1).

Effects on the slave SH2. The slave mixes 8 channels of 8-bit samples straight from ROM and feeds the 32X PWM: mono, cycle 1044 (~22 kHz), control `0x0185`. The master sends play/stop/volume commands through a small ring in SDRAM, always accessed through the cache-through alias (address | `0x20000000`): each SH2 has its own cache, and neither may see a stale copy. This needs no Sega CD and adds no COMM traffic.

Music as CD audio.

- Tyrian's AdLib songs were rendered offline with the game's own player and OPL emulator, one loop per song, 44.1 kHz stereo: 41 tracks, 46.5 minutes.
- Song *n* is track *n* + 2 (track 1 is data). Fix the mapping in one place and log every request in the PC build.
- Normalise loudness when rendering: the first render played at half the expected volume. A uniform gain of 2.0–2.5 with a soft limiter above 24000 fixed it.
- The 68000 starts tracks with BIOS play-repeat (CMD 45) and stops them (CMD 54), as in Kobo. Without a Sega CD both commands are acknowledged and ignored, so the cartridge still runs silently.
- Volume control and fades are not available for CD audio here; treat a fade as a stop.

Toolchain and build hygiene

Most lost test cycles came from testing a build that was not the one just made. These habits removed that whole class of mistake.

- **Never leave an old ROM behind.** The build deletes the previous image first and prints a loud "BUILD FAILED – NO ROM WAS WRITTEN" banner with the failing line.
- **Rebuild on content, not timestamps.** On a Windows drive under WSL, unzipping or copying can make old object files look newer than changed sources, so `make` kept stale code. The build hashes the sources and rebuilds from scratch when the hash changes.

- **Show the build on screen.** The title screen prints the build time and mode (`32X mm-dd hh:mm`, plus `PROF`/`HOT`). A screenshot or video then says exactly which code ran.
- **One job at a time on `/mnt`.** Parallel builds hit random "Permission denied" errors while Windows or Defender held files open.
- **Use newlib, carefully.** Unlike Kobo's library-free SH2 build ([KOBO-BOOT] §6.8), Tyrian links newlib for `malloc` and `snprintf`. That makes a large C codebase portable, but every library call in a hot path must be checked (previous section).
- **Let the compiler check foreign interfaces.** Tyrian's 68000 code includes D32XR's `scd.c` into its own file, so a prototype mismatch with D32XR becomes a build error instead of a crash.
- **Derive the header from the build.** Entry points, VBRs, copy size and heap bounds come from ELF symbols; the cartridge builder refuses an image whose pieces do not line up.

On-screen diagnostics. Kobo's diagnostic strip and status codes ([KOBO-LEDGER], [KOBO-BOOT] §3 stage 7) are the model. Tyrian added a frame profiler (milliseconds per part of the game loop, averaged over 32 frames) and red/blue error screens showing the faulting PC. Kobo's warning holds: on-screen text is expensive ([KOBO-BOOT] §6.7); measure the overlay's own cost and subtract it.

The PC host build as a test bench

A PC build that runs the exact port code against the exact ROM image caught most bugs before any emulator run. It replaces only the platform layer (`plat_host.c` instead of `plat_mars.c`); the game, the file system, the renderer and the memory tricks are the same code.

What made it useful.

- **Determinism.** A virtual clock and a fixed random seed make every run repeatable. All five of Tyrian's demos must stay pixel-identical after any change; that one test caught most regressions.
- **Scripted pad input.** `frame:buttons` scripts drive scenarios such as title → shop → level 1 → game over → title.
- **Faithful hardware quirks.** The two frame buffers swap at every flip, as on the 32X; the heap is counted per block with newlib-like overhead; a 32-bit build matches SH2 pointer sizes for struct layouts.
- **Shared code across CPUs.** The CD directory and window code (`cd_files.c`) is compiled for the 68000 and for the PC. The PC reads sectors from the real disc image, either the ISO or the raw 2352-byte BIN, so the disc builder and the parser are tested together.
- **Self-checks against the original.** Tables baked at build time (items, texts) are compared field by field with the game's own parser on every PC run.
- **Audio to a file.** The same mixer writes a WAV of what the 32X should play, which was later confirmed by ear on the console.

Its limits. It cannot show bus contention, frame-buffer access cost, DMA speed or torn COMM reads. On the PC the DMA is a `memcpy`, so a DMA path that was slower on the 32X looked fine there. Timing questions need the on-screen profiler, and hardware rules need the console ([KOBO-BOOT] §6).

Checklist for the next builder

Before writing code

- ☐ Choose CD boot or cartridge + CD (section 2), based on whether the read-only data fits 4 MB of ROM.
- ☐ Measure the game's RAM needs on a PC build first; list what can be used in place, baked at build time, or allocated on demand.
- ☐ Claim command numbers in the shared registry and write a per-phase owner for every COMM register.
- ☐ Decide where music comes from (CD audio, PCM, PWM) and what each CPU does; give the slave SH2 a job.

While porting

- ☐ Build the PC host bench early: deterministic clock, scripted pad, frame dumps, heap report, a pixel-identical regression test.
- ☐ Keep the 68000's polling loops in Genesis RAM.
- ☐ Write the frame buffer in 16/32-bit units; draw each pixel once; draw straight into the back buffer.
- ☐ Disassemble hot paths for hidden library calls.
- ☐ Put a build id on the title screen and a profiler behind a build switch.
- ☐ Follow every rule in [KOBO-BOOT] §6: read flags twice, pause between polls, no data reads during music without restarting it, black as `0x0400` when Genesis graphics show through.

Before release

- ☐ Test on real hardware, not only Fusion; ares for cartridge + CD.
- ☐ Test without a Sega CD, with a foreign disc in the drive, and with no SRAM.
- ☐ Test long sessions for stack overflow and heap fragmentation.

Open hardware questions from the Tyrian work

- Does a real 32X stall SH2 ROM accesses while RV is set, or return garbage? Tyrian assumes the worst and parks both SH2s.
- Do Fusion, ares and the EverDrive all honour cartridge SRAM reached through RV on a 4 MB 32X cartridge?
- D32XR's `scd_read_sectors` signature and its Word RAM addresses (`0x0C0000` Sub side, `0x600000` 68000 side in Mode 1) were taken from notes; confirm against the D32XR version you build with.
- The COMM streaming rate on real hardware has not been measured; the design target was a few hundred KB/s.